Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу

«Операционные системы»

Группа: М8О-209БВ-25

Студент: Выскворкин А. В.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 14.10.26

Москва, 2026

Постановка задачи

Вариант 7.

Составить программу на языке Си, обрабатывающую данные в многопоточном режиме. Использовать стандартные средства создания потоков операционной системы (Windows/Unix).

Два человека играют в кости. Правила игры следующие: каждый игрок делает бросок 2-ух костей K раз; побеждает тот, кто выбросил суммарно большее количество очков. Задача программы экспериментально определить шансы на победу каждого из игроков. На вход программе подаётся K, какой сейчас тур, сколько очков суммарно у каждого из игроков и количество экспериментов, которые должна произвести программа.

Общий метод и алгоритм решения

Использованные системные вызовы:

- ssize_t write(int fd, const void* buf, size_t count); – записывает count байт из buf в файл с дескриптором fd.
- void exit(int status); – завершает процесс с кодом возврата status.
- int clock_gettime(clockid_t clock_id, struct timespec *tp); – записывает в tp текущее значение времени часов clock_id
- int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine)(void *), void *arg); – создаёт новый поток в процессе. Идентификатор потока записывается в *thread. Поток начинает выполнение функции start_routine(), с аргументами для функции *arg.
- int pthread_join(pthread_t thread, void **retval); – ждёт завершения потока thread.
- int pthread_mutex_lock(pthread_mutex_t *mutex); – если мьютекс разблокирован, поток блокирует мьютекс. Если мьютекс заблокирован, поток ждёт разблокировки.
- int pthread_mutex_unlock(pthread_mutex_t *mutex) – разблокирует мьютекс.

Число потоков задаётся последним аргументов программы. Каждый поток получает свой сид для генерации результатов бросков кубиков. Количество экспериментов делится поровну между потоками. Если число экспериментов не делится нацело на число потоков, оставшиеся n экспериментов распределяются между первыми n потоками. Каждый поток проводит в цикле нужно число экспериментов, сохраняя в локальные счётчики побед первого и второго игрока. Для избежания гонки данных при прибавлении результата к глобальным счётчикам, используется мьютекс. После завершения всех потоков выводится результат.

Код программы

sequencial.c

```
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

typedef struct {
    size_t first_score;
    size_t second_score;
    size_t round;
```

```
    size_t K;
} GameState;

static size_t dice_roll(int* seed) {
    return (size_t)(rand_r(seed) % 6 + 1);
}

static int DefineWinner(int* seed, const GameState* game) {
    size_t scoreOne = game->first_score;
    size_t scoreTwo = game->second_score;
    for (size_t r = game->round; r <= game->K; r++) {
        scoreOne += dice_roll(seed) + dice_roll(seed);
        scoreTwo += dice_roll(seed) + dice_roll(seed);
    }
    if (scoreOne > scoreTwo)
        return 1;
    if (scoreOne < scoreTwo)
        return 2;
    return 0;
}

int main(int argc, char** argv) {
    struct timespec start, end;
    clock_gettime(CLOCK_MONOTONIC, &start);

    if (argc != 6) {
        char msg[1024];
        int len = snprintf(msg, sizeof(msg) - 1, "usage: %s K round_number first_points
second_points experiment_number\n", argv[0]);
        write(STDERR_FILENO, msg, len);
        exit(EXIT_SUCCESS);
    }

    GameState game = {
        .first_score = (size_t)strtoull(argv[3], NULL, 10),
        .second_score = (size_t)strtoull(argv[4], NULL, 10),
        .K = (size_t)strtoull(argv[1], NULL, 10),
        .round = (size_t)strtoull(argv[2], NULL, 10)
    };
    size_t exp_num = (size_t)strtoull(argv[5], NULL, 10);

    if (game.K < game.round || game.K == 0 || game.round == 0 || exp_num == 0 ||
        game.first_score < 2*(game.round - 1) || game.second_score < 2*(game.round - 1) ||
        game.first_score > 12*(game.round - 1) || game.second_score > 12*(game.round - 1)) {
            char msg[] = "Invalid input";
            write(STDERR_FILENO, msg, sizeof(msg) - 1);
            exit(EXIT_FAILURE);
    }
```

```
    size_t first_win_count = 0;
    size_t second_win_count = 0;
    int seed = (int)time(NULL);

    for (size_t i = 0; i < exp_num; i++) {
        int winner = DefineWinner(&seed, &game);
        if (winner == 1)
            first_win_count++;
        else if (winner == 2)
            second_win_count++;
    }

    double first_P = (double)first_win_count / (double)exp_num * 100;
    double second_P = (double)second_win_count / (double)exp_num * 100;

    char result[1024];
    int len = snprintf(result, sizeof(result) - 1, "First wins: %.2f%%\n", first_P);
    write(STDOUT_FILENO, result, len);
    len = snprintf(result, sizeof(result) - 1, "Second wins: %.2f%%\n", second_P);
    write(STDOUT_FILENO, result, len);

    clock_gettime(CLOCK_MONOTONIC, &end);
    int run_time = (int)(((end.tv_sec + end.tv_nsec / 1e9) - (start.tv_sec + start.tv_nsec / 1e9)) *
1000);
    len = snprintf(result, sizeof(result) - 1, "Time: %d ms\n", run_time);
    write(STDOUT_FILENO, result, len);

    return 0;
}
```

parallel.c

```
#include <unistd.h>
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

static pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

typedef struct {
    size_t first_score;
    size_t second_score;
    size_t round;
    size_t K;
    size_t exp_num;
    size_t n_threads;
} GameState;
```

```
typedef struct {
    size_t number;
    int seed;
} ThreadArgs;

static GameState game;
static size_t first_win_count = 0;
static size_t second_win_count = 0;

static size_t dice_roll(int* seed) {
    return (size_t)(rand_r(seed) % 6 + 1);
}

static int DefineWinner(int* seed, const GameState* game) {
    size_t scoreOne = game->first_score;
    size_t scoreTwo = game->second_score;
    for (size_t r = game->round; r <= game->K; r++) {
        scoreOne += dice_roll(seed) + dice_roll(seed);
        scoreTwo += dice_roll(seed) + dice_roll(seed);
    }
    if (scoreOne > scoreTwo)
        return 1;
    if (scoreOne < scoreTwo)
        return 2;
    return 0;
}

static void *work(void* _args) {
    ThreadArgs *args = (ThreadArgs *)_args;

    size_t my_exp_num = game.exp_num / game.n_threads;
    size_t exp_remain = game.exp_num % game.n_threads;
    if (args->number < exp_remain)
        my_exp_num++;

    size_t my_first_wins = 0;
    size_t my_second_wins = 0;
    int my_seed = args->seed;

    for (size_t i = 0; i < my_exp_num; i++) {
        int winner = DefineWinner(&my_seed, &game);
        if (winner == 1)
            my_first_wins++;
        else if (winner == 2)
            my_second_wins++;
    }
```

```
    pthread_mutex_lock(&mutex);
    first_win_count += my_first_wins;
    second_win_count += my_second_wins;
    pthread_mutex_unlock(&mutex);

    return NULL;
}

int main(int argc, char** argv) {
    struct timespec start, end;
    clock_gettime(CLOCK_MONOTONIC, &start);

    if (argc != 7) {
        char msg[1024];
        int len = snprintf(msg, sizeof(msg) - 1, "usage: %s K round_number first_points
second_points experiment_number n_threads\n", argv[0]);
        write(STDERR_FILENO, msg, len);
        exit(EXIT_SUCCESS);
    }

    game.K = (size_t)strtoull(argv[1], NULL, 10);
    game.round = (size_t)strtoull(argv[2], NULL, 10);
    game.first_score = (size_t)strtoull(argv[3], NULL, 10);
    game.second_score = (size_t)strtoull(argv[4], NULL, 10);
    game.exp_num = (size_t)strtoull(argv[5], NULL, 10);
    game.n_threads = (size_t)strtoull(argv[6], NULL, 10);

    if (game.K < game.round || game.K == 0 || game.round == 0 || game.exp_num == 0 ||
        game.first_score < 2*(game.round - 1) || game.second_score < 2*(game.round - 1) ||
        game.first_score > 12*(game.round - 1) || game.second_score > 12*(game.round - 1) ||
        game.n_threads < 1) {
            char msg[] = "Invalid input";
            write(STDERR_FILENO, msg, sizeof(msg) - 1);
            exit(EXIT_FAILURE);
    }

    pthread_t* threads = malloc(game.n_threads * sizeof(pthread_t));
    ThreadArgs* thread_args = malloc(game.n_threads * sizeof(ThreadArgs));

    int seed = (int)time(NULL);
    for (size_t i = 0; i < game.n_threads; i++) {
        thread_args[i] = (ThreadArgs){.number = i, .seed = seed + i};
        pthread_create(&threads[i], NULL, work, &thread_args[i]);
    }

    for (size_t i = 0; i < game.n_threads; i++)
        pthread_join(threads[i], NULL);
```

```
    double first_P = (double)first_win_count / (double)game.exp_num * 100;
    double second_P = (double)second_win_count / (double)game.exp_num * 100;

    char result[1024];
    int len = snprintf(result, sizeof(result) - 1, "First wins: %.2f%%\n", first_P);
    write(STDOUT_FILENO, result, len);
    len = snprintf(result, sizeof(result) - 1, "Second wins: %.2f%%\n", second_P);
    write(STDOUT_FILENO, result, len);

    clock_gettime(CLOCK_MONOTONIC, &end);
    int run_time = (int)(((end.tv_sec + end.tv_nsec / 1e9) - (start.tv_sec + start.tv_nsec / 1e9)) *
1000);
    len = snprintf(result, sizeof(result) - 1, "Time: %d ms\n", run_time);
    write(STDOUT_FILENO, result, len);

    free(thread_args);
    free(threads);

    return 0;
}
```

Протокол работы программы

Число потоков	Время выполнения (мс)	Ускорение	Эффективность
1	13135	-	-
2	6646	1,976	0,988
3	4528	2,9	0,967
4	3494	3,759	0,94
12	2113	6,216	0,518
1024	1929	6,809	0,0066
8192	2072	6,339	0,00077
32768	2690	4,883	0,00015

Ускорение вычисляется как $S = \frac{T_s}{T_p}$, где T_s – время выполнения последовательного кода, T_p – время выполнения параллельного кода.

Эффективность вычисляется как $x = \frac{S}{p}$, где p – количество ядер.

```
huawei@LAPTOP-L1DAUAII:/mnt/d/OS/Lab2$ ./seq 10 1 0 0 100000000
First wins: 48.14%
Second wins: 48.13%
Time: 13135 ms
huawei@LAPTOP-L1DAUAII:/mnt/d/OS/Lab2$ ./par 10 1 0 0 100000000 2
First wins: 48.15%
Second wins: 48.14%
Time: 6646 ms
huawei@LAPTOP-L1DAUAII:/mnt/d/OS/Lab2$ ./par 10 1 0 0 100000000 3
First wins: 48.15%
Second wins: 48.15%
Time: 4528 ms
huawei@LAPTOP-L1DAUAII:/mnt/d/OS/Lab2$ ./par 10 1 0 0 100000000 4
First wins: 48.16%
Second wins: 48.16%
Time: 3494 ms
huawei@LAPTOP-L1DAUAII:/mnt/d/OS/Lab2$ ./par 10 1 0 0 100000000 12
First wins: 48.15%
Second wins: 48.15%
Time: 2113 ms
huawei@LAPTOP-L1DAUAII:/mnt/d/OS/Lab2$ ./par 10 1 0 0 100000000 1024
First wins: 48.15%
Second wins: 48.15%
Time: 1929 ms
huawei@LAPTOP-L1DAUAII:/mnt/d/OS/Lab2$ ./par 10 1 0 0 100000000 8192
First wins: 48.15%
Second wins: 48.16%
Time: 2072 ms
huawei@LAPTOP-L1DAUAII:/mnt/d/OS/Lab2$ ./par 10 1 0 0 100000000 32768
First wins: 48.15%
Second wins: 48.16%
Time: 2690 ms
```

Вывод

В результате выполнения лабораторной работы были приобретены практические навыки в управлении потоками ОС и обеспечением их синхронизированной работы. Для синхронизации потоков использовался примитив синхронизации мьютекс. Исследована зависимость между ускорением, эффективностью и числом потоков. На малом числе потоков (1 - 4) ускорение близко к линейному, т. к. каждый поток независимо проводит своё количество экспериментов. Единственная критическая секция – прибавление к глобальным счётчикам побед значение локальных счётчиков потоков. При количестве потоков, сильно превышающих число физических ядер (1024, 8192, 32768) эффективность падает, так как при увеличении числа потоков уменьшается процессорное время для каждого, увеличиваются расходы на переключение между потоками.